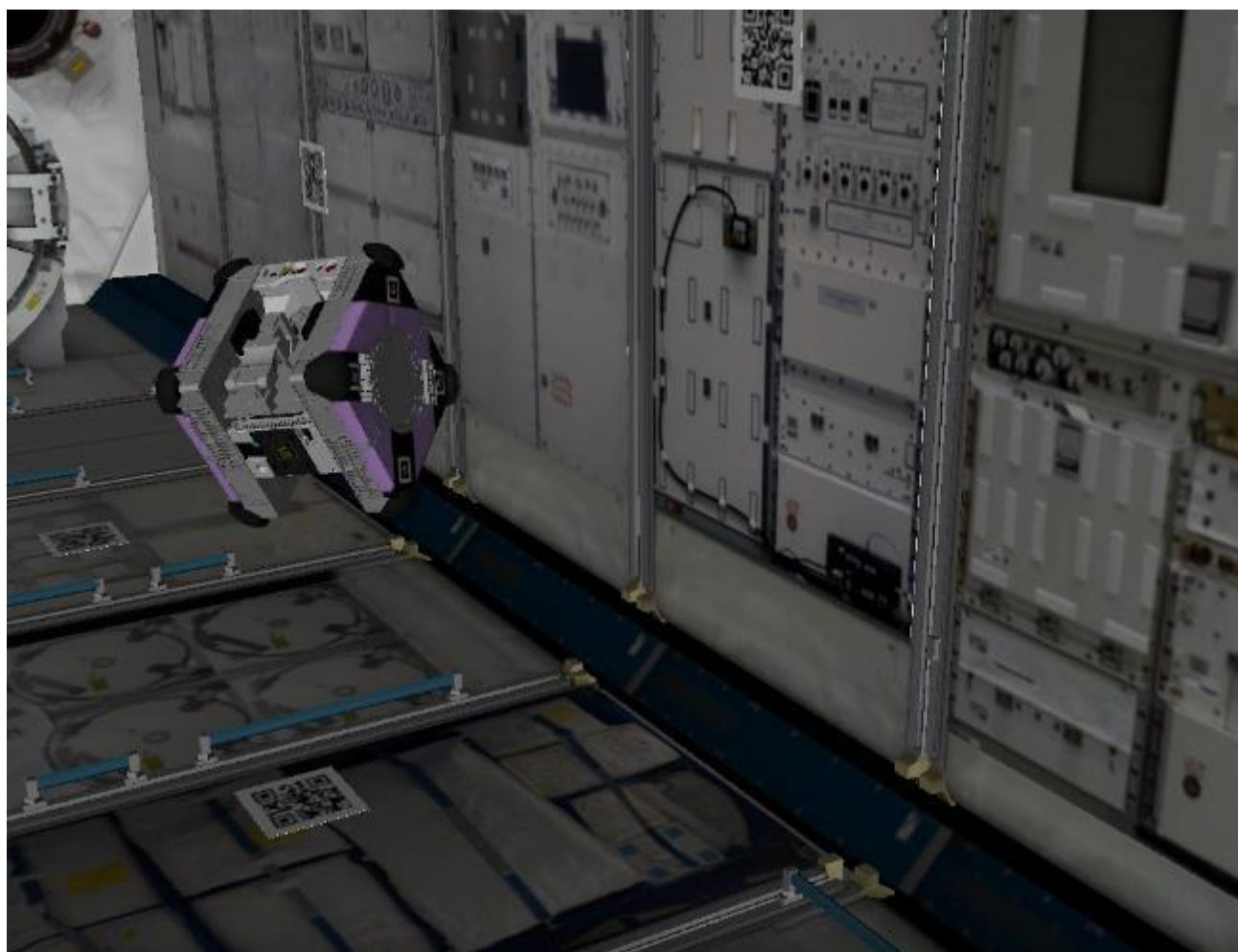


6th Kibo Robot Programming Challenge Programming Manual



Version 3.0 (Release Date: Oct. 15th, 2025)

Japan Aerospace Exploration Agency (JAXA)

General Point of Contact: Z-KRPC@ml.jaxa.jp

List of Changes

All changes to paragraphs, tables, and figures in this document are shown below;

Release Date	Revision	Paragraph(s)	Rationale
April 1 st , 2025	1.0	All	-
May 27 th , 2025	2.0	Update “5. Programming tips”	5.12. Library for Image Processing • Added opencv:4.9.0 description. 5.13. Library for Object Detection • Added information about the OpenCV DNN module. 5.14. Conversion of Object Detection Models Other Than ‘tf lite’ Format • New section added.
October 15 th , 2025	3.0	Update “5. Programming tips”	5.16. 【For the Final Round】 About the Noise Simulation Module • New section added. 5.17. 【For the Final Round】 About the Gravity Simulation Module • New section added.



Contents

1. Introduction	1
2. Setting up your machine.....	2
2.1. Requirements	2
2.2. Setting up Android Studio.....	2
3. Creating your application.....	4
3.1. Creating an Android project.....	4
3.2. Writing the application.....	5
3.3. Building your application	7
4. Running your program on the simulator.....	12
4.1. Using the simulator server	12
4.2. Login.....	12
4.3. Uploading the APK and running your program	14
4.4. Checking simulation while running	15
4.5. Checking the result	15
4.6. Running on your own machine (optional).....	23
5. Programming tips	34
5.1. Do NOT write infinite loops	34
5.2. Debugging feature for image processing.....	35
5.3. Dealing with randomness.....	35
5.4. About navigation errors	37
5.5. Flashlight	37
5.6. Error handling	37
5.7. Attention to computing resources	38
5.8. Performance of Localization.....	39
5.9. Code review.....	41
5.10. Setting the application ID	41
5.11. Questions and information exchange	41
5.12. Library for Image Processing	41
5.13. Library for Object Detection	42
5.14. Conversion of Object Detection Models Other Than 'tflite' Format	42
5.15. Adding Libraries	43
5.16. 【For the Final Round】 About the Noise Simulation Module	43
5.17. 【For the Final Round】 About the Gravity Simulation Module	45
6. Game API details.....	46
6.1. Method summary	46
6.2. Method details	47
7. Tips for Astrobee Characteristics	53
7.1. Rendering of Astrobee	53



7.2. Specification of Astrobeer.....55

7.3. References of Astrobeer55



1. Introduction

Let's start programming!

Astrobee can be controlled with an Android application called the Guest Science APK (GS APK). First, setup your machine to build your application according to the instructions in Chapter 2. Next, read Chapter 3 and create a GS APK. Then, try running the GS APK in the simulator environment. Chapter 4 describes how to use the simulator environment.



2. Setting up your machine

First of all, set up a machine for programming.

2.1. Requirements

The machine must meet the following requirements.

- 64-bit processor
- 4 GB RAM (8 GB RAM recommended)
- Ubuntu 20.04 (64-bit version) (<http://releases.ubuntu.com/20.04/>)
or Windows 10 (64-bit version) or Windows 11 (64-bit version)

NOTE: If you want to run your program on your own PC, you need 16 GB of RAM and Ubuntu 20.04. For details, please refer to 4.6. Running on your own machine (optional).

2.2. Setting up Android Studio

2.2.1. Installing components (Only on Ubuntu)

If you use an Ubuntu machine, you need these components.

- openJDK8
- ADB (Android Debug Bridge)
- Gradle

Please install them with the following command.

```
sudo apt-get -y install openjdk-8-jdk adb gradle
```

2.2.2. Installing Android Studio

Please download Android Studio 3.6.3 from the Android Studio download archives page (<https://developer.android.com/studio/archive>) and extract it to your home directory.



2.2.3. Downloading additional components

To build the Guest Science APK, you need to download additional components as follows.

- 1) Launch Android Studio.
- 2) Select [Tools] -> [SDK Manager].
- 3) On the SDK Platforms tab, check "Show Package Details" and select "Android SDK Platform 25", "Android SDK Platform 26", and "Android SDK Platform 28".
- 4) On the SDK Tools Tab, check "Show Package Details" and select "25.0.3", "26.0.2", "28.0.3" under Android SDK Build-Tools and select "20.0.5594570" under NDK (Side by side)."
- 5) Click the [Apply] button to install these components.

3. Creating your application

3.1. Creating an Android project

To create your application, prepare a new project with the following steps.

- 1) Download the APK (Sample APK or Template APK) from the download page on the web site.
- 2) Extract the zip file to the directory where you want it.
- 3) Launch Android Studio.
- 4) Open the APK template folder with [File] -> [Open].
- 5) Open [app/java/jp.jaxa.iss.kibo.rpc.sampleapk/YourService.java] or [app/java/jp.jaxa.iss.kibo.rpc.defaultapk/YourService.java] in Project view.
- 6) Write your code in runPlan1 – runPlan3 methods in the YourService.java file.

When you open the APK template folder, the “Android Gradle Plugin Update Recommended” dialog may appear. However, you must not update the plugin because of a dependency problem, so select “Don’t remind me again for this project.”

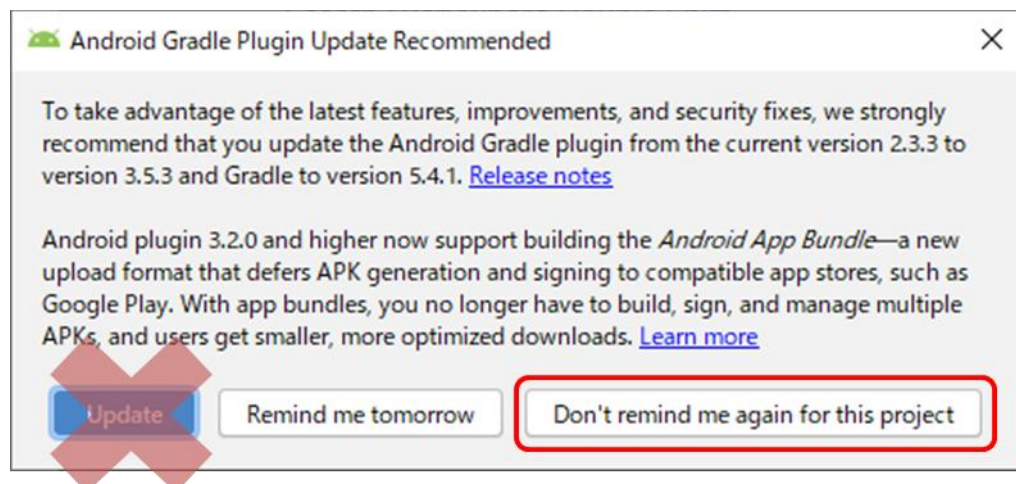


Figure 3-1 Android Gradle Plugin Update Recommended dialog



3.2. Writing the application

You can use the game APIs shown below in the YourService.java file.

“runPlan1” is executed on the web simulator. You can choose any plan when you run the application on your own machine.

```
public class YourService extends KiboRpcService {
    @Override
    protected void runPlan1(){
        // The mission starts.
        api.startMission();

        // Move to a point.
        Point point = new Point(10.9d, -9.92284d, 5.195d);
        Quaternion quaternion = new Quaternion(0f, 0f, -0.707f, 0.707f);
        api.moveTo(point, quaternion, false);

        // Get a camera image.
        Mat image = api.getMatNavCam();

        /* ***** */
        /* Write your code to recognize type and number of landmark items in the each area! */
        /* If there is a treasure item, remember it. */

        /* ***** */

        // When you recognize landmark items, let's set the type and number.
        api.setAreaInfo(1, "item_name", 1);

        /* ***** */
        /* Let's move to the each area and recognize the items. */
        /* ***** */

        // When you move to the front of the astronaut, report the rounding completion.
        point = new Point(11.143d, -6.7607d, 4.9654d);
        quaternion = new Quaternion(0f, 0f, 0.707f, 0.707f);
        api.moveTo(point, quaternion, false);
        api.reportRoundingCompletion();

        /* ***** */
        /* Write your code to recognize which target item the astronaut has. */
        /* ***** */
    }
}
```



```

        // Let's notify the astronaut when you recognize it.
        api.notifyRecognitionItem();

        /*
        ***** */

        /* Write your code to move Astrobee to the location of the target item (what the astronaut
        is looking for) */
        /*
        ***** */

        // Take a snapshot of the target item.
        api.takeTargetItemSnapshot();
    }
    protected void runPlan2() {
        // You can write your other plan here, but it's not run on the web simulator.
        // ...
    }
}

```

You can find methods of the game APIs by using the code completion function of Android Studio.

Please refer to 6. Game API details for more information and you can download a sample APK from the download page on the web site.

NOTE: Please create applications using the recommended libraries as much as possible. Non-recommended libraries may work in the simulator but may not function on the real robot.

If you use non-recommended libraries, please be aware of the differences in processor architecture. Additionally, if it is determined during the review process that the application may not work, you will be asked to modify the application.



3.3. Building your application

3.3.1. On Ubuntu

To build your application, use the command shown below.

NOTE: DO NOT build your application using Android Studio to change the build task, as this may cause an error.

```
$ cd <YOUR_APK_PATH>
$ ANDROID_HOME=$HOME/Android/Sdk ./gradlew assembleDebug
```

You can find the APK file here: "<YOUR_APK_PATH>/app/build/outputs/apk/debug/app-debug.apk".

3.3.2. On Windows

Please build your application with the following steps.

- 1) Launch the Android Studio.
- 2) Open <YOUR_APK_PATH>.
- 3) Click app on the [Project] window.
- 4) Select [Build] -> [Make Module 'app'].

If you find errors, please build an APK file on an Ubuntu machine.

You can find the APK file here: "<YOUR_APK_PATH>\app\build\outputs\apk\debug\app-debug.apk".

3.3.3. Change the application ID, APK name and short name (optional)

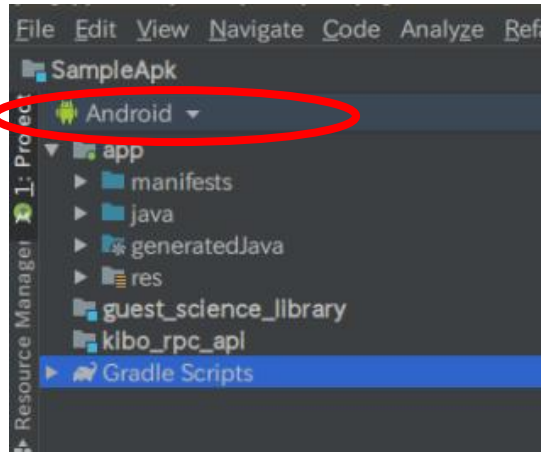
You can change the application ID (*jp.jaxa.iss.kibo.rpc.sampleapk* or *jp.jaxa.iss.kibo.rpc.defaultapk* by default).

In this step, we change the application ID to "jp.jaxa.iss.kibo.rpc.myteam" and the APK name/short name to "myteam" with the SampleAPK project.

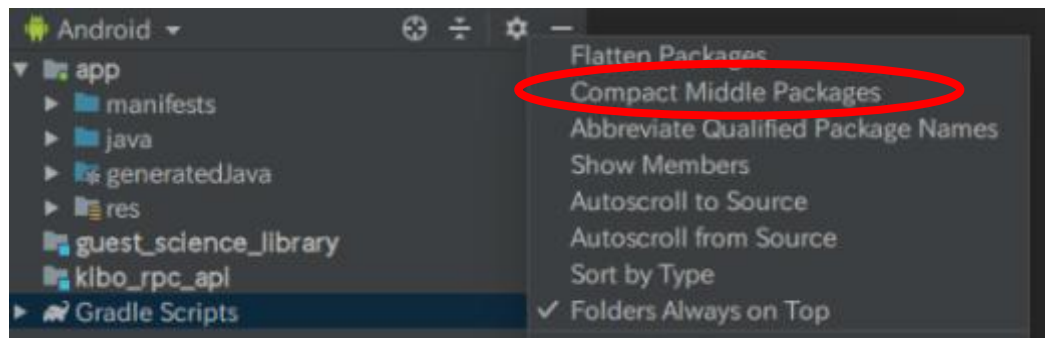
NOTE: This instruction is for the Final Round. Changing the application ID is not necessary and not recommended in the Preliminary Round.

NOTE: "jp.jaxa.iss.kibo.rpc" cannot be changed.

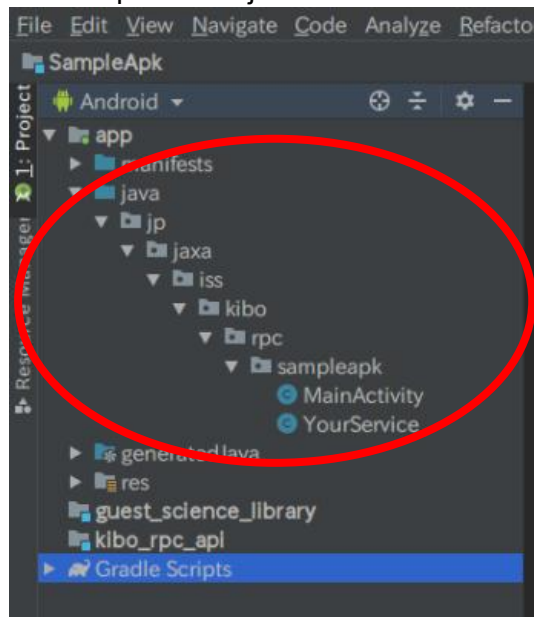
- 1) Launch Android Studio.
- 2) Open <YOUR_APK_PATH>.
- 3) Make sure you are viewing the project in Android View.

**Figure 3-2 Android View**

- 4) Click on settings (the gear icon) and deselect [Compact Middle Package].

**Figure 3-3 Unselect [Compact Middle Package]**

- 5) Please expand the “java” folder.

**Figure 3-4 Expand the “java” folder**

- 6) Right-click the “sampleapk” folder and select [refactor] -> [rename].

- 7) A warning will be displayed, but you want to go ahead and click [Rename Package]. After that, enter the APK name that you want. (In the picture, we rename as it “myteam”.)

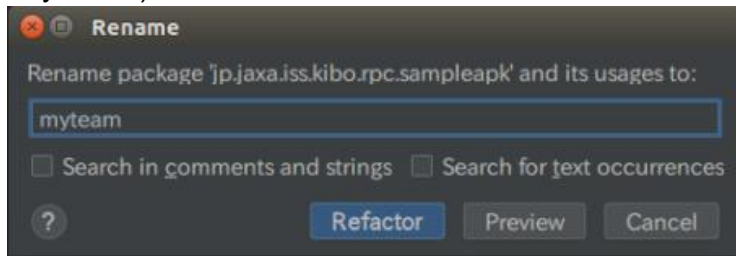


Figure 3-5 Rename dialog

- 8) At the bottom of Android Studio, “Refactoring Preview” will be displayed. Here, click [Do Refactor].

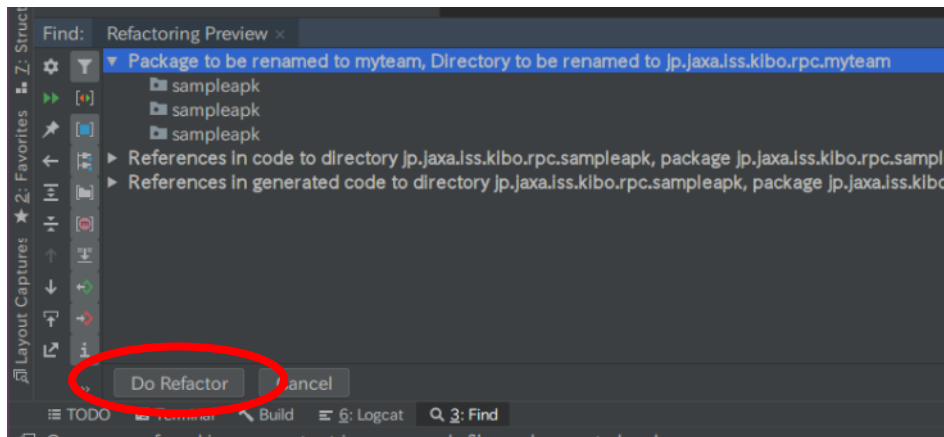


Figure 3-6 Refactoring Preview

- 9) Open **build.gradle (Module: app)** in Gradle Scripts on the left-side of the menu. Please change the application ID and click [Sync Now].

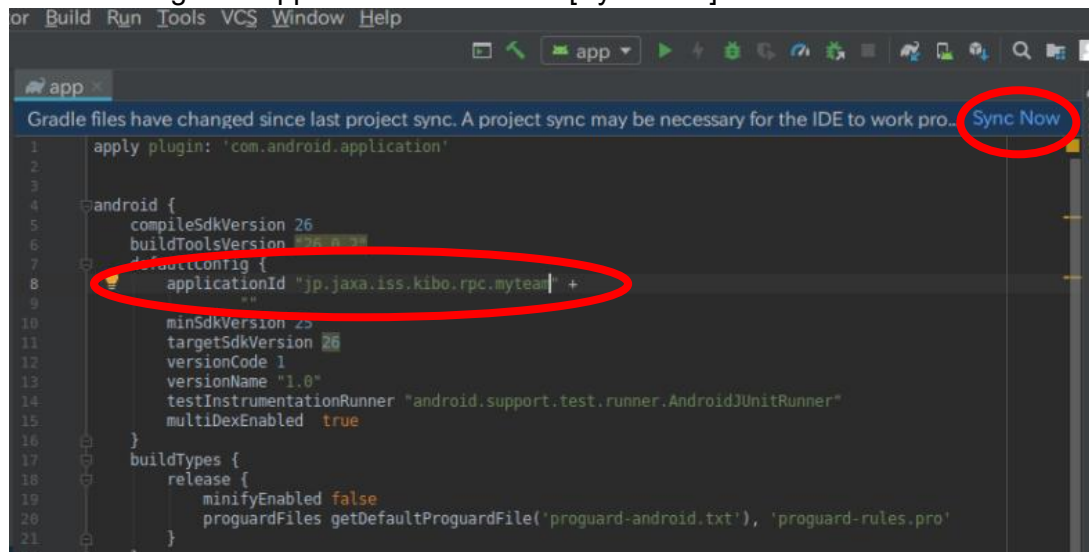


Figure 3-7 build.gradle (Module: app)

- 10) Open **strings.xml** in app -> src -> main -> res -> values on the left-side of the menu. Please change the APK name and save it.

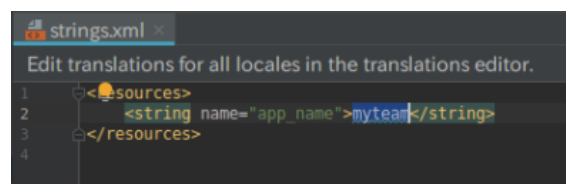


Figure 3-8 strings.xml

- 11) Open **commands.xml** in app -> src -> main -> res -> xml on the left-side of the menu. Please change the value of "shortName" tag and save it. If there is no "shortName" tag, please add it under the "apkInfo" tag and set your short name.

```
<shortName>myteam</shortName>
```



Figure 3-9 commands.xml

You have successfully changed the application ID in Android Studio. If you want to change the Android project name and its directory name, follow the following steps.



■ On Ubuntu

- 12) Close Android Studio.
- 13) Please execute the following commands.

```
cd <YOUR_APK_PATH>
cd ../
mv SampleApk <YOUR_APK_NAME>
cd <YOUR_APK_NAME>
mv SampleApk.iml <YOUR_APK_NAME>.iml
```

■ On Windows

- 12) Close the Android Studio.
- 13) Please rename a **SampleApk** folder to <YOUR_APK_NAME> with Windows Explorer.
- 14) Now rename **SampleApk.iml** to <YOUR_APK_NAME>.iml in the **SampleApk** folder with Windows Explorer.

4. Running your program on the simulator

4.1. Using the simulator server

Once you have built your application, you can run it on the web simulator provided by JAXA. To use the simulator, you need a user account issued by the Kibo-RPC secretariat. If you don't have one, please read the Kibo-RPC Guidebook to complete your application for participating in Kibo-RPC first.

Note that the actual Kibo environment is not exactly the same as the simulation environment, since there are many objects in the Kibo and the environment changes frequently. Please refer to the image and Google Map for the actual environment. (These are not the latest.)



Figure 4-1 Image inside Kibo

Google Map: https://www.google.com/maps/@29.5604024,-95.0855631,2a,75y,205.79h,103.61t/data=!3m6!1e1!3m4!1sUA46_vlbk9kAAQvxgbyMg!2e0!7i10000!8i5000

Also, the views of Astrobe's NavCam and DockCam are different between the simulator and the real robot. Refer to the image below.

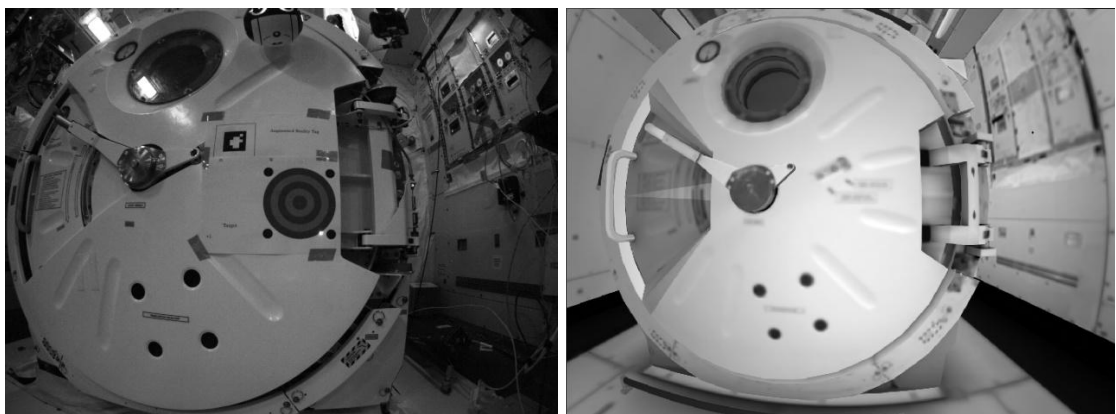


Figure 4-2 (Left) Actual snapshot (Right) Simulator image

4.2. Login

Access the Kibo-RPC web site (<https://jaxa.krpc.jp/>) and click "LOGIN."



Figure 4-3 LOGIN tab

On the login page, enter the ID and password for your team's account, and click the "LOGIN" button. If you have forgotten your ID, please contact the Kibo-RPC secretariat. You can reset your password by clicking the "Forgot your Password?" link.

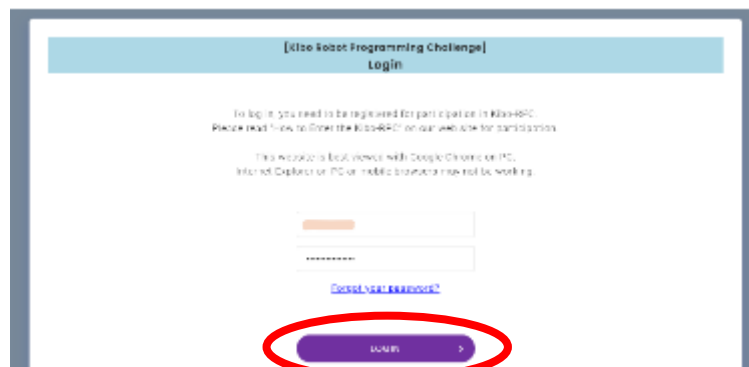


Figure 4-4 LOGIN button

On the home page, click "SIMULATION."



Figure 4-5 SIMULATION button

Now, you can access the web simulator from this page.

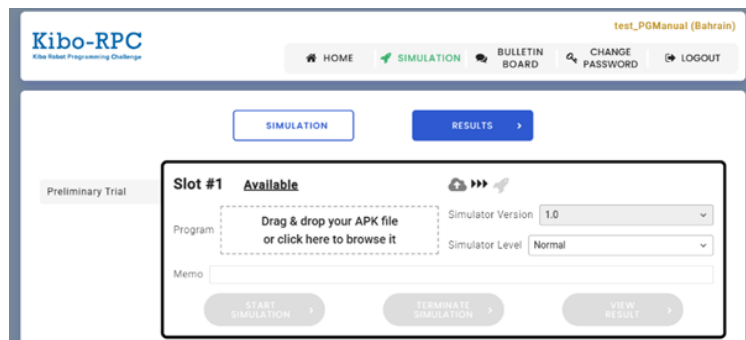


Figure 4-6 Web simulator page

4.3. Uploading the APK and running your program

On the simulation page, there are three slots for simulations, so that you can run at most three programs in parallel.

To start your simulation, select your APK file, the simulator version, and the simulation conditions.

Enter a memo if desired, and click the “START SIMULATION” button.

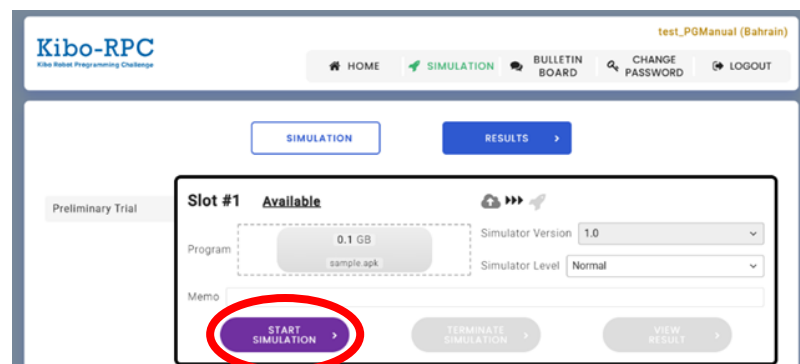


Figure 4-7 START SIMULATION button

A simulation may take longer than 20 minutes to run, and it does not need your attention while it runs. After starting your simulation, you can log out, get a cup of coffee, then go back to the web site.

When there is a simulation running, the slot displays its original information, and you cannot run another simulation in the same slot until it finishes.

If you want to stop your simulation, click the “TERMINATE SIMULATION” button. Note that terminating a simulation loses its game score and output files (such as rosbag and the Android Emulator’s log).

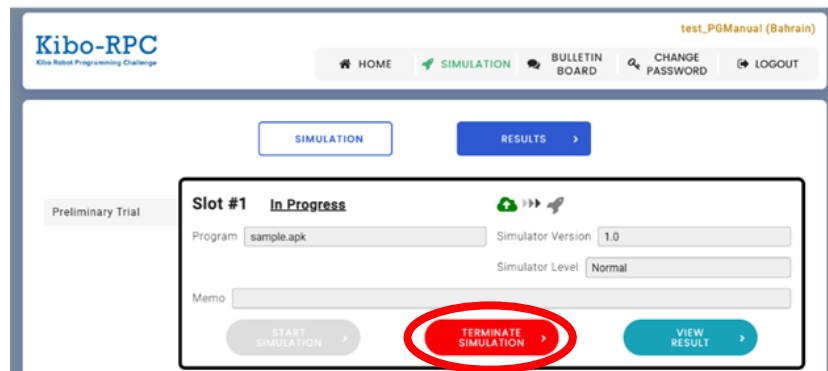


Figure 4-8 TERMINATE SIMULATION button

4.4. Checking simulation while running

When your simulation is running, you can log in to the simulator server (viewer) via your browser. Click the “SIMULATOR VIEWER” button to show the information for a remote connection and open the viewer in another tab by clicking the “VIEW” button.

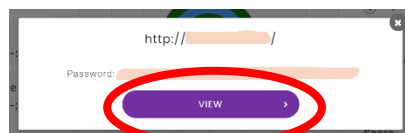


Figure 4-9 VIEW button

Enter the password for your remote connection to log in. Now you can use rviz to see how Astrobees move in your simulation. This viewer is available until the simulation is finished.

The viewer displays a real-time simulation in the view-only mode for the simulation stability. You cannot operate the viewer.

4.5. Checking the result

4.5.1. Result summary

Once your simulation has started, you can check the results by clicking the “VIEW RESULT” button on the simulation page.



Figure 4-10 VIEW RESULT button

On the result page, you can see the details of your simulation, such as the game time, the matching of areas and items, and so on.

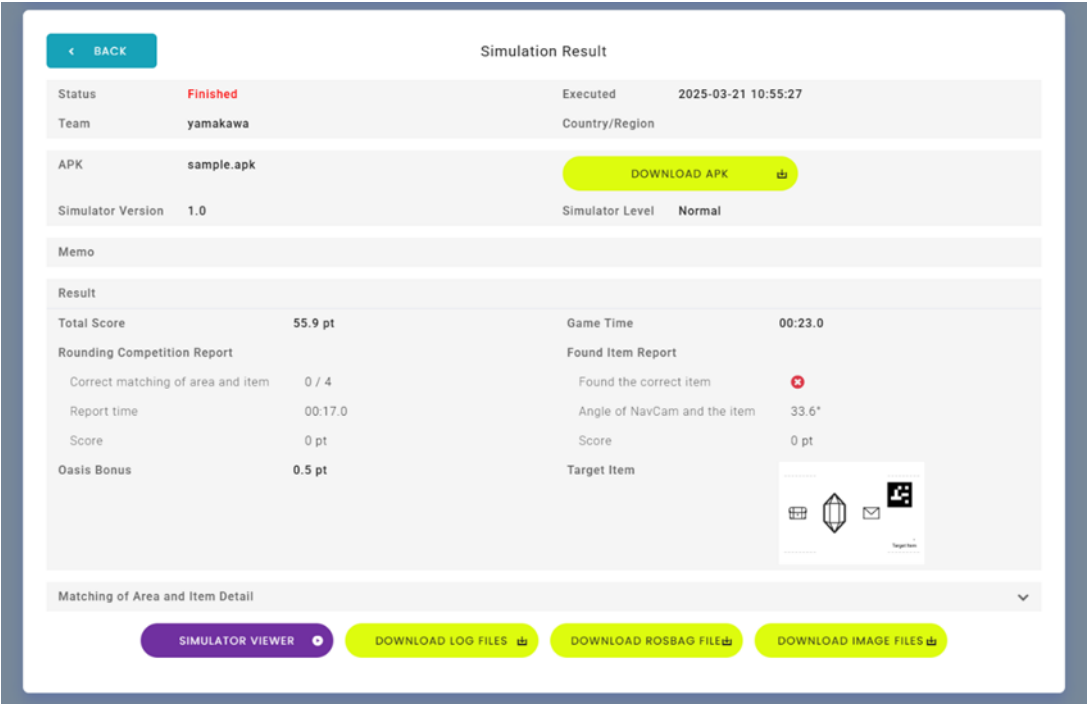


Figure 4-11 RESULT page

Table 4-1 Report Result

Found the correct item	Green check	Astrobees found the lost item the astronaut was looking for at the correct position within the time limit.
	Red cross	Astrobees executed takeTargetItemSnapshot API at a wrong position. Or Astrobee did not try to execute takeTargetItemSnapshot API.

**Refer to the rulebook for the defined correct position.*

Click “Matching of Area and Item Detail”, detail of mapping information is displayed.

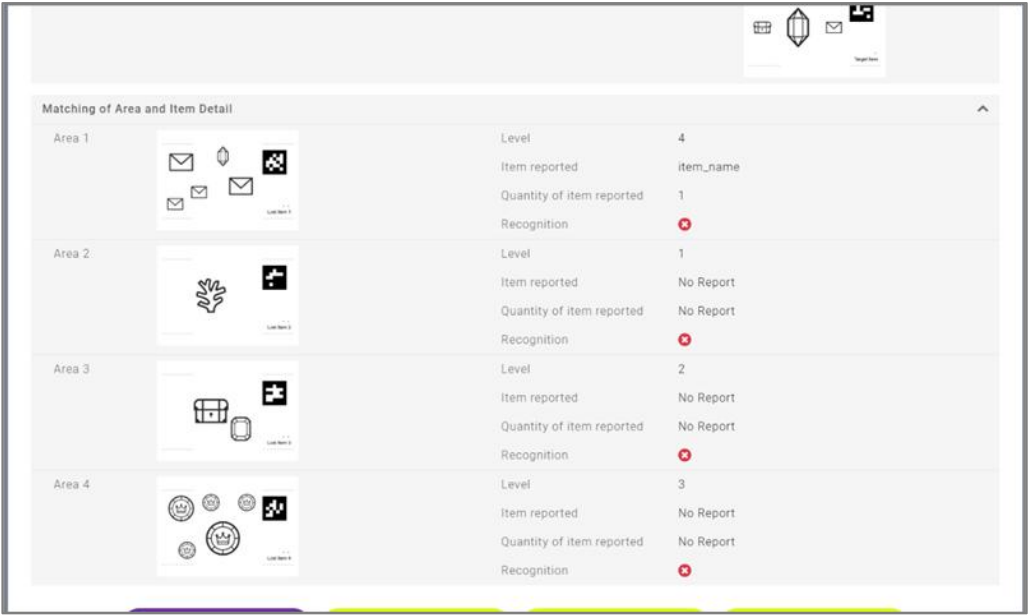


Figure 4-12 Matching of Area and Item Detail

Table 4-2 Target Result

Recognition	Green check	The type and number of Items reported were correct.
	Red cross	The type and number of Items reported were wrong.



4.5.2. Download ZIP file

You can get a ZIP file by clicking the “DOWNLOAD LOG FILES” button. This ZIP file contains the game score and the Android Emulator’s log. The output log level is INFO. Note that some or all of these files will not be available unless your simulation finishes properly. Besides the result page, the game score also appears in a JSON file, which can be read using a text editor.

Table 4-3 Example of result.json

```
{
  "Mission Time": {
    "start": "20240123 203208003",
    "finish": "20240123 203456003"
  },
  "RoundingReport": {
    "timestamp": "20240123 203456003",
    "arrival": {
      "try": true,
      "success": true,
      "distance": 3.2
    },
    "areaMapping": {
      "areas": [
        {
          "areald": 1,
          "level": 1,
          "correct": true,
          "lost_item": "item_name",
          "num": 1
        },
        {
          "areald": 2,
          "level": 3,
          "correct": false,
          "lost_item": "item_name",
          "num": 1
        },
        {
          "areald": 3,
          "level": 2,
          "correct": true,
          "lost_item": "item_name",
          "num": 1
        }
      ]
    }
  }
}
```

“Mission Time” is the difference between the “start” time and the “finish” time.

“try” is true if you execute reportRoundingCompletionAPI. “success” is true if you execute reportRoundingCompletionAPI at the correct position.

“correct” is true if both “lost_item” and “num” are correct.

The string for “lost_item” needs to match the string described in the rulebook.



```

        "areald": 4,
        "level": 4,
        "correct": true,
        "lost_item": " item_name",
        "num": 1
    }
]
},
"FoundReport": {
    "timestamp": "20210315 203323123",
    "cameraAngle": 45,
    "distance": 3.2,
    "try": true,
    "isInArea": true,
    "success": true
},
"illegal": false,
"OasisZoneN": 300,
"TimeBonusN": 0.1
}

```

"try" is true if you execute
takeTargetItemSnapshotAPI.
"success" is true if you execute
takeTargetItemSnapshotAPI at the correct
position.

"illegal" is for server internal use.

You can also get a rosbag as a ZIP file by clicking the "DOWNLOAD ROSBAG FILE" button. The size of this file will be so large that it may take a long time to download it.

In addition, you can save any debug images with saveBitmapImage/saveMatImage API and download the images by clicking the "DOWNLOAD IMAGE FILES" button.

4.5.3. Check simulation after running

To check previous simulations, click the “Results” button on the simulation page. The results page lists your past simulations. This list can hold up to 20 simulations.

Executed	Status	Score	Memo	
2022-03-27 10:45:23	Finished	30.06 pt		VIEW REMOVE SUBMIT
2022-03-27 10:22:35	Finished	00.00 pt		VIEW REMOVE SUBMIT
2022-03-27 10:04:54	Finished	30.25 pt		VIEW REMOVE SUBMIT
2022-03-27 09:43:30	Finished	00.00 pt		VIEW REMOVE SUBMIT

Figure 4-13 Results list page

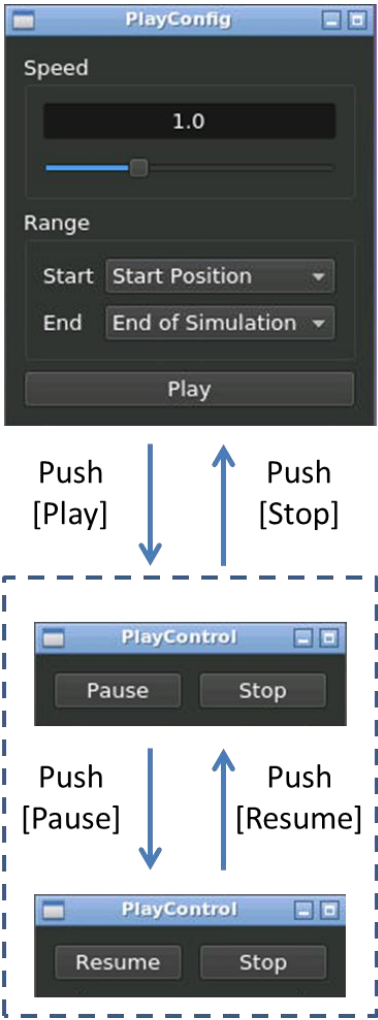
The “VIEW RESULT” button is the same as the one on the simulation page. Please be careful when you click the “REMOVE RESULT” button; it removes the output files of the selected simulation and these results will be lost.

You can play the rosbag (simulation result) at 0.5x – 3x speed with the viewer. You can change rosbag replay settings and rviz settings. The details are, described in the sections below.



4.5.4. rosbag replay settings

You can change rosbag replay settings using Rosbag Player.



Type	Description
Speed Slider	Select replay speed.
Range Selector	Select replay range.
Play Button	Start replay and open rviz window. If rviz already has opened, it will restart.
Pause Button	Pause replay.
Resume Button	Resume replay.
Stop Button	Stop replay and back to PlayConfig window.

Figure 4-14 Rosbag Player

4.5.5. rviz settings

You can change the display settings for the rviz window.

Table 4-4 rviz configuration

Item	Check box in the “Displays” tab
Planning trajectory	[Visualize]->[PlanningTrajectory]
Trajectory	[Visualize]->[Trajectory]
KeepInZone/KeepOutZone	[Visualize]->[Zones]
NavCam	[Sensors]->[NavCam]
DockCam	[Sensors]->[DockCam]
HazCam	[Sensors]->[HazCam]

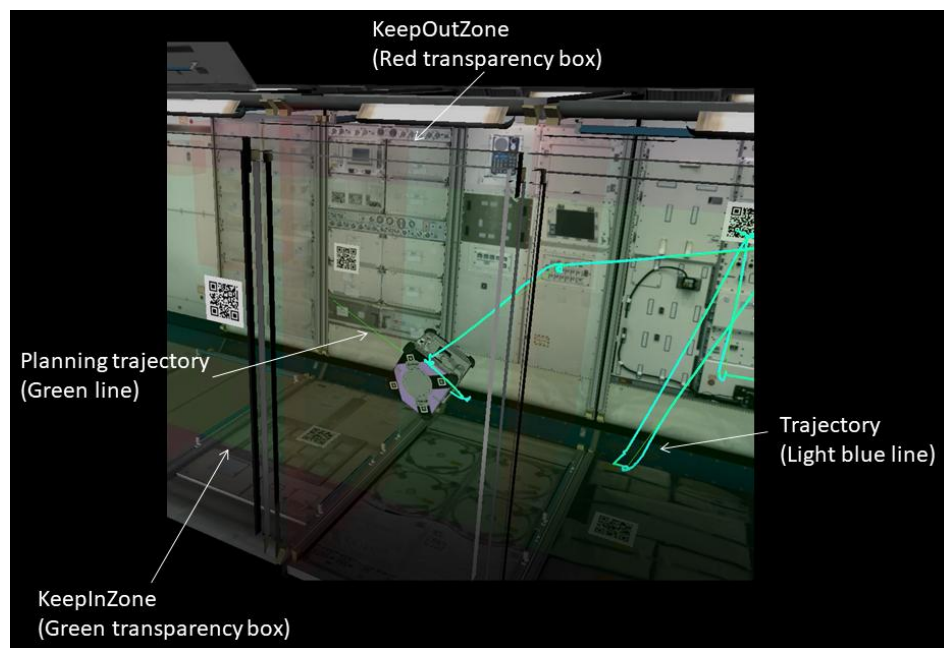


Figure 4-15 rviz configuration description



4.6. Running on your own machine (optional)

You can also run the program on your own machine. This chapter provides a procedure to set up the Astrobee Simulator. You get a simple simulation environment without randomness modules (Phases/targets randomness generator, air flow simulator and navigation errors) or a judge module.

4.6.1. Differences between web simulator and local simulator

Local Simulation Environment do not include random factor modules (object type and position, airflow, and navigation error).

You can test and debug your program using a local simulator, but you need to evaluate it on a web simulator server in order to obtain a high score in the Preliminary Round.

4.6.2. Requirements

The following requirements are needed to set up a simulation environment on your machine.

- 64-bit processor
- 16 GB RAM
- Ubuntu 20.04 (64-bit version) (<http://releases.ubuntu.com/20.04/>)
- Disk: 30GB of free space (SSD is highly recommended)

4.6.3. Overview

The overall procedure is as follows.

1. Installing Docker
2. Building the Kibo-RPC Simulator with Docker
3. Setting Android Emulator to run APK
4. Building the Guest Science Manager APK
5. Setting up the network to connect the Android emulator and the simulator
6. Installing your APK to Android Emulator
7. Launching the simulator and run your APK
 - i. Launching the Android Emulator
 - ii. Starting the Kibo-RPC Simulator
 - iii. Running the Guest Science Manager, GDS Simulator and your APK

4.6.4. Installing Docker

Please install Docker engine before setting the simulator.

```
$ sudo apt-get update
$ sudo apt-get install ca-certificates curl gnupg lsb-release
$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/docker-archive-keyring.gpg
$ echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-
  keyring.gpg] https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
$ sudo apt-get update
$ sudo apt-get install docker-ce docker-ce-cli containerd.io
$ sudo groupadd docker
$ sudo usermod -aG docker $USER
```

Then, logout and log back in.

For more details, see official instructions:

<https://docs.docker.com/engine/install/ubuntu/>

<https://docs.docker.com/engine/install/linux-postinstall/>

4.6.5. Building the Kibo-RPC Simulator with Docker

Download the Kibo-RPC Simulator Setting up scripts from our website.

(<https://jaxa.krpc.jp/download>)

This module contains the following scripts.

- build.sh
- run.sh

The build.sh clones the Astrobe Robot software from GitHub, applies patches of Kibo-RPC modules and builds docker images.

All you have to do is to execute the script as follows.

Note that the building sequence will take several hours to complete and needs large disk space (20-30 GB).

※Builds may fail under certain conditions. When the build fails, check the CPU and memory utilization, delete unnecessary processes, and re-run the build.

```
unzip krpc6_simulator.zip -d $HOME
cd ${HOME}/local_simulator
docker load -i krpc_astrobeesim_local.tar
```

Once building the docker images is finished, you can launch the simulator.

Edit hosts file to set up the network.

```
sudo nano /etc/hosts
```

Add the following three lines and save the file.

```
127.0.0.1      hlp
127.0.0.1      mlp
127.0.0.1      llp
```

If hlp, mlp, and llp settings already exist, comment them out, add three new lines, and save them.

```
#10.42.0.36    hlp
#10.42.0.35    mlp
#10.42.0.34    llp

127.0.0.1      hlp
127.0.0.1      mlp
127.0.0.1      llp
```

Finally, you can run a simulation.

```
bash run.sh
```

Is the image below displayed on your screen? If so, installation is complete!

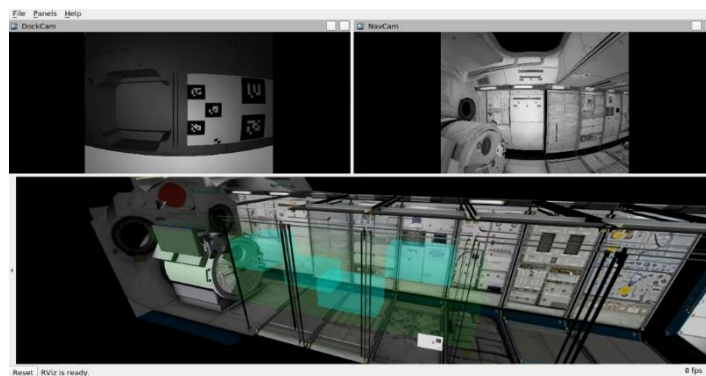


Figure 4-16 Setup result



The following error will appear, but this is because the device is not connected. Please ignore this error as the simulator will work normally.

```
[ERROR] [1742302049.035362566, 7.632000000] : (ros.depth_odometry)
image_features_with_known_correspondences_aligner_depth_odometry.cc:87:
DepthImageCallback: Time difference too large, time diff: 1.336
```

Type Ctrl+C on the terminal to stop the simulator, and proceed to the next step.

4.6.6. Setting the Android Emulator to run APK

Create an AVD (Android Virtual Device) as follows.

- 1) Launch Android Studio.
- 2) Select [Tools] -> [AVDManager].
- 3) In the Android Virtual Device Manager window, click [+ Create Virtual Device ...].
- 4) Select device **Nexus 5** (Resolution 1080x1920) and click [Next].
- 5) Select the [x86 Images] tab, choose **Nougat/API Level 25/ABI x86_64/Android 7.1.1(NO Google APIs)**, then click [Next].

NOTE: Download the system image now if you need it.

- 6) Set the AVD name to “AstrobeeAndroidSim.”
- 7) Click [Finish].

In the Android Virtual Device Manager window, you will see “AstrobeeAndroidSim” in the list.

Click the Play button in the Action column. If the AVD launches successfully, you will see the following image.

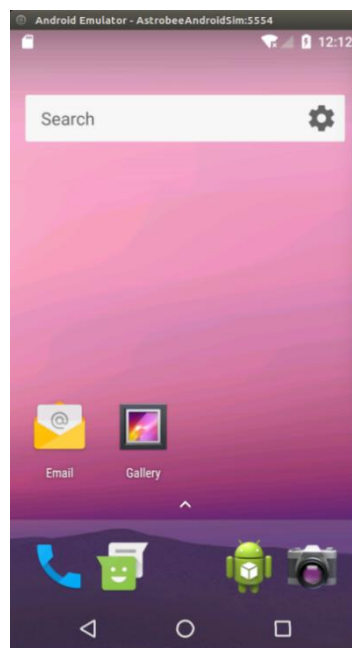


Figure 4-17 Android Emulator screen

See https://github.com/nasa/astrobee_android/blob/master/emulator.md for details.

Shutdown the emulator and let's proceed to the next step.



4.6.7. Building the Guest Science Manager APK

To run your program, you must install the Guest Science Manager APK. (see details at https://github.com/nasa/astrobee_android/blob/master/guest_science_readme.md)

At first, clone the Astrobee Android package from GitHub.

```
export ANDROID_PATH=${HOME}/astrobee_android
git clone https://github.com/nasa/astrobee\_android.git
git checkout a8560ab0270ac281d8eadeb48645f4224582985e
```

Next, build the Guest Science Manager APK.

Modify the Guest Science Manager build.gradle as follows:

- Modify the Guest Science Manager build.gradle as follows.

```
vi $ANDROID_PATH/core_apks/guest_science_manager/build.gradle
```




- Replace the contents of build.gradle with the following.

```
// Top-level build file where you can add configuration options common to all sub-
// projects/modules.

buildscript {
    repositories {
        mavenCentral()
    }
    dependencies {
        classpath 'com.android.tools.build:gradle:2.3.0'

        // NOTE: Do not place your application dependencies here; they belong
        // in the individual module build.gradle files
    }
}

allprojects {
    repositories {
        maven {
            url "https://github.com/rosjava/rosjava_mvn_repo/raw/master"
        }
        maven {
            url "https://raw.githubusercontent.com/rosjava/rosjava_mvn_repo/master/"
        }
        maven {
            url "https://artifacts.camunda.com/artifactory/public/"
        }
        mavenLocal()
        mavenCentral()
    }
}

task clean(type: Delete) {
    delete rootProject.buildDir
}
```

- Run the build.

```
cd $ANDROID_PATH/core_apks/guest_science_manager
ANDROID_HOME=$HOME/Android/Sdk ./gradlew assembleDebug
```

4.6.8. Setting up the network

Setup the network between the Kibo-RPC Simulator and the Android Emulator.

See https://github.com/nasa/astrobee_android/blob/master/emulator.md for details.

■ Setting the HOST network

Execute the following command to open the hosts file.

```
sudo nano /etc/hosts
```

Comment out the previous three lines, add new three lines below and save the hosts file.

```
#127.0.0.1    hlp
#127.0.0.1    mlp
#127.0.0.1    llp

10.42.0.36    hlp
10.42.0.35    mlp
10.42.0.34    llp
```

■ Setting the environment variables

Execute the following commands to set the environment variables.

```
export ANDROID_PATH=$HOME/astrobee_android
export EMULATOR=$HOME/Android/Sdk/tools/emulator
export AVD="AstrobeAndroidSim"
```

Note that you need to execute the above commands whenever you open a terminal. If you write these commands in your `.bashrc` file, you don't have to execute them.

■ Setting up the Android network and starting the Android Emulator

Execute the following commands to set up the Android network and launch the Android Emulator.

```
cd $ANDROID_PATH/scripts
./launch_emulator.sh -n
```

See https://github.com/nasa/astrobee_android/blob/master/emulator.md for details.

* If the Android network does not work, try turning off Wi-Fi in the Android Emulator.

4.6.9. Installing APKs

If the Android Emulator is not running, execute the following commands to start it.

```
cd $ANDROID_PATH/scripts  
./launch_emulator.sh -n
```

In another terminal, execute the following commands to install the Guest Science Manager APK and your GS APK.

```
cd $ANDROID_PATH/core_apks/guest_science_manager  
adb install -g -r activity/build/outputs/apk/activity-debug.apk  
cd <YOUR_APK_PATH>  
adb install -g -r app/build/outputs/apk/debug/app-debug.apk
```

4.6.10. Running your program

It's time to run your program!

■ Launching the Android Emulator

Execute the following commands to launch the Android Emulator if it is not running.

```
cd $ANDROID_PATH/scripts  
./launch_emulator.sh -n
```

■ Starting the Kibo-RPC Simulator

Execute the following command to start the Kibo-RPC Simulator.

```
cd ${HOME}/local_simulator  
bash run.sh
```

■ Running the Guest Science Manager, GDS Simulator and your GS APK

Execute the following commands to start the Guest Science Manager APK and to launch the GDS simulator.

Note: It's required to re-execute these commands whenever you re-launch the simulator or re-install your APK.

```
ANDROID_PATH/scripts/gs_manager.sh start
```

```
docker exec -it astrobee bash  
cd /src/astrobee/src/tools/gds_helper/src  
python3 gds_simulator.py
```

Operate the GDS simulator to run your GS APK.

- 1) Press any key to grab control.
- 2) Select your GS APK.
- 3) Type **b** and press **Enter** to start the GS APK.
- 4) Type **d** and press **Enter** to send a custom guest science command.

Now Astrobee starts to locate the item!

※ ***When the GDS Simulator times out***

The following steps may solve the problem.

■ Launching the Android Emulator

Execute the following commands to launch the Android Emulator.

```
cd $ANDROID_PATH/scripts  
./launch_emulator.sh
```

■ Starting the Kibo-RPC Simulator

Execute the following command to start the Kibo-RPC Simulator.

```
cd ${HOME}/local_simulator  
bash run.sh
```

■ Running the Guest Science Manager, GDS Simulator and your GS APK



Execute the following commands to start the Guest Science Manager APK and to launch the GDS simulator.

```
adb shell su 0 sh /cache/emulator_setup_net.sh
bash $ANDROID_PATH/scripts/gs_manager.sh start
docker exec -it astrobee bash
cd /src/astrobee/src/tools/gds_helper/src
python3 gds_simulator.py
```



5. Programming tips

5.1. Do NOT write infinite loops

You **must not** write any infinite loops in your code because no one can stop Astrobee while the loop is executing.

Double check that you use finite loops with a defined counter value, as shown below.

```
// NG
while(!result.hasSucceeded()){
    // do something
}

// OK
final int LOOP_MAX = 5;
int loopCounter = 0;
while(!result.hasSucceeded() && loopCounter < LOOP_MAX){
    // do something
    ++loopCounter;
}
```

5.2. Debugging feature for image processing

You can save any Bitmap/Mat type images in the Android Emulator and download the images from the dashboard display. This feature should be useful to check intermediate images of your image processing algorithm.

To save an image, use `saveBitmapImage` or `saveMatImage` API as follows.

```
// get/process a bitmap image
Bitmap image = any_function();
// save the image
api.saveBitmapImage(image, "file_name_1");

// get/process a mat image
Mat img = any_function_mat();
// save the image
api.saveMatImage(img, "file_name_2");
```

If you are running APK on your local machine, the image can be obtained by the following command.

```
(If your APK is based on TemplateAPK)
adb pull /sdcard/data/jp.jaxa.iss.kibo.rpc.defaultapk/immediate/DebugImages

(If your APK is based on Sample APK)
adb pull /sdcard/data/jp.jaxa.iss.kibo.rpc.sampleapk/immediate/DebugImages
```

Up to 50 images can be saved per simulation, and the maximum image size is 1228800 pixels (1280 x 960 px).

5.3. Dealing with randomness

You must consider the randomness of the environment.

When you want to move the robot, refer to the commands below...

```
// move to point 1
api.moveTo(point1, quaternion1, true);
// move to point 2
api.moveTo(point2, quaternion2, true);
// move to point 3
api.moveTo(point3, quaternion3, true);
```

If there is no randomness in the environment, this code works well.



However, Astrobees may be faced with errors such as **tolerance violations** and **collision detection (*)**, and your code will not work, so you have to provide redundant code, as we see below.

Remember, **Do NOT** allow any infinite loops in your code!

* Tolerance violation error occurs when there is a discrepancy between Astrobees's pose (estimated) and the target pose. Collision detection occurs when Astrobees's HazCam detects any obstacles on the target path. Both errors can occur for a variety of causes including false detection, especially in the real environment.

```
Result result;
final int LOOP_MAX = 5;

// move to point 1(first try)
result = api.moveTo(point1, quaternion1, true);

// check result and loop while moveTo api is not succeeded.
// Do NOT write infinite loop.
int loopCounter = 0;
while(!result.hasSucceeded() && loopCounter < LOOP_MAX){

    // retry
    result = api.moveTo(point1, quaternion1, true);
    ++loopCounter;

}
// move to point 2
//...
```


5.4. About navigation errors

The real world always has uncertainties. Navigation error is one of them and the Kibo-RPC simulator server simulates it.

However, modeling and simulating navigation errors are highly complicated, and this increases the calculation load. Therefore, a random error following Gaussian distribution is used generally.

The Kibo-RPC simulator also implements a Gaussian distribution and the parameters are as follows;

Regarding position;

x: mean = 0 m and 3sigma = 0.1 m

y: mean = 0 m and 3sigma = 0.1 m

z: mean = 0 m and 3sigma = 0.1 m

Regarding orientation;

x: mean = 0 degree and 3sigma = 3 degree

y: mean = 0 degree and 3sigma = 3 degree

z: mean = 0 degree and 3sigma = 3 degree

You have to consider that self-positioning and self-orientation obtained from the APIs (getRobotKinematics) includes these errors.

5.5. Flashlight

You must consider the effect of light conditions on the image recognition in the real environment. In our past experiments, when the brightness value was set to 0.05 or higher and taken a snapshot in front of the image, the light reflected on the illustration and the image could not be recognized properly.

```
// turn on the front flashlight
api.flashlightControlFront(0.01f);
// you might need some sleep...
Thread.sleep(2000);
// get a camera image
Mat image = api.getMatNavCam();
// turn off the front flashlight
api.flashlightControlFront(0.0f);
```

5.6. Error handling

Various errors can occur due to external and internal causes during program run. Considering all kinds of situations and handling the errors make your program more robust to environmental changes.

Null check: Ensure variable is not null.

```
Mat image = api.getMatNavCam();
```



```
// You must handle null pointer
If (image == null) {
    /** Error handling**/
} else {
    readAR(image);
}
```

Size of list/array: Check the length of list/array when access its element.

```
List<String> list = getStringList();

// Bad
for (int i=0; i<5; i++) {
    // this code cause an exception when the list size is less than 5.
    String str = list.get(i);
}

// Good
for (int i=0; i<list.size(); i++) {
    String str = list.get(i);
}
```

try-catch: Handle exceptions with try-catch statements.

```
try {
    String decodedStr = someDecoder();
    int value = Integer.parseInt(decodedStr);
    // ...
} catch (NumberFormatException e) {
    /** Handle the exception occurs when the string is not number **/
} catch (Exception e) {
    /** Handle unexpected exceptions to prevent program down **/
}
```

5.7. Attention to computing resources

If the computing loads are high, Astrobeer might be overloaded and not work on orbit. The specifications of Astrobeer real robot's HLP are as follows. Note that available resources are different by other software working on HLP.

Multithreading is not recommended, but in our past experiments, up to two simultaneous threads can work without problems.

CPU: Qualcomm Snapdragon 820 (4 cores, 2.2GHz)

RAM: 4GB

5.8. Performance of Localization

There is a possibility of losing Astrobees' self-position on-orbit. Once the self-position is lost, Astrobees may not be able to recover on its own. In this case, it means it is a game over. Be aware that this incident is not in the simulator. It is well known that the technology Astrobees uses is likely to lose its self-position when the navigation camera view gets too close to wall, floor, airlock, and so on because the camera cannot capture enough features at those places. Please note the above when creating your program.

For more information, mapped landmark (ML) features coverage heat map is shown in the figures below.

The localization performance depends on how many ML features NavCam captures. The heat map represents the number of mapped landmark (ML) features in a volume of 30 cm³ along an imaginary grid on the overhead, aft, forward, deck walls and the front of the airlock.

The more ML features in a given volume the more stable Astrobees localization will be.

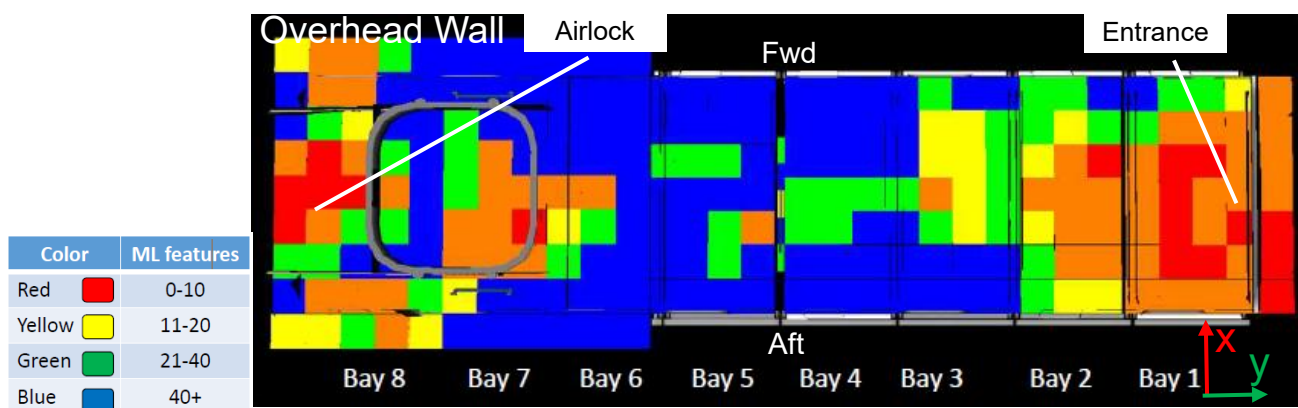


Figure 5-1 ML features on overhead wall

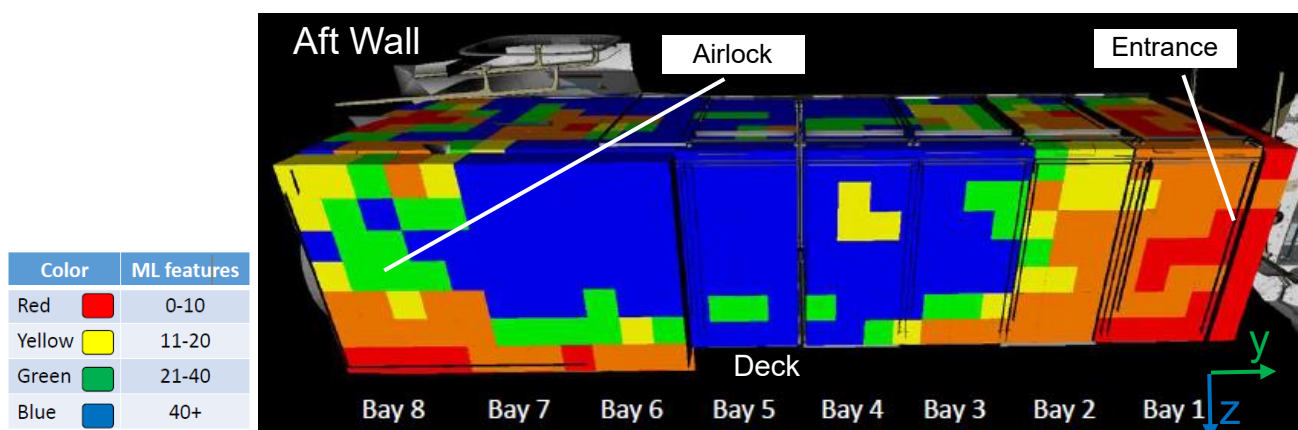


Figure 5-2 ML features on aft wall

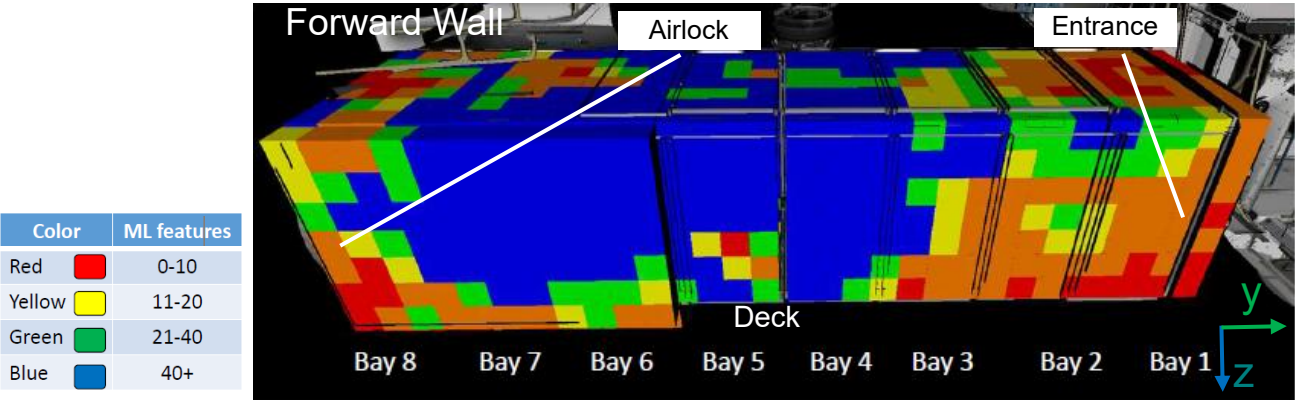


Figure 5-3 ML features on fwd wall

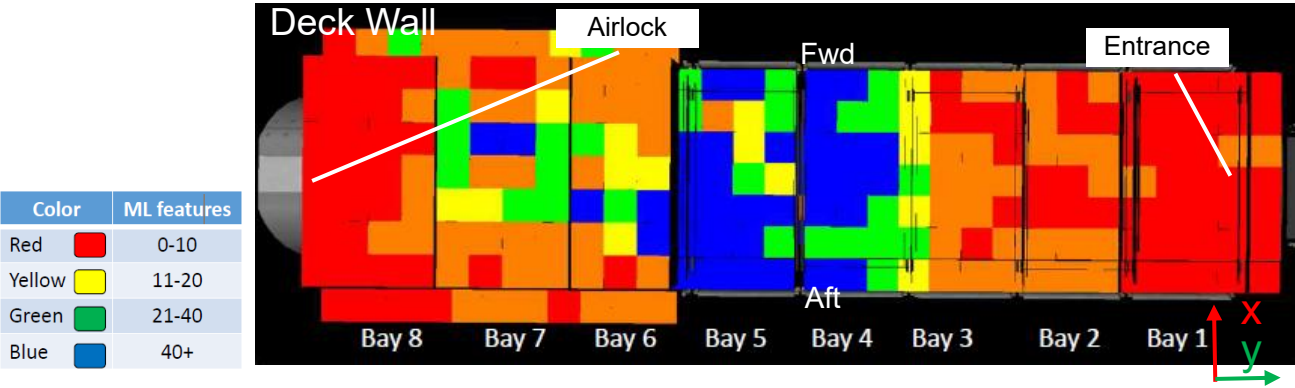


Figure 5-4 ML features on deck wall

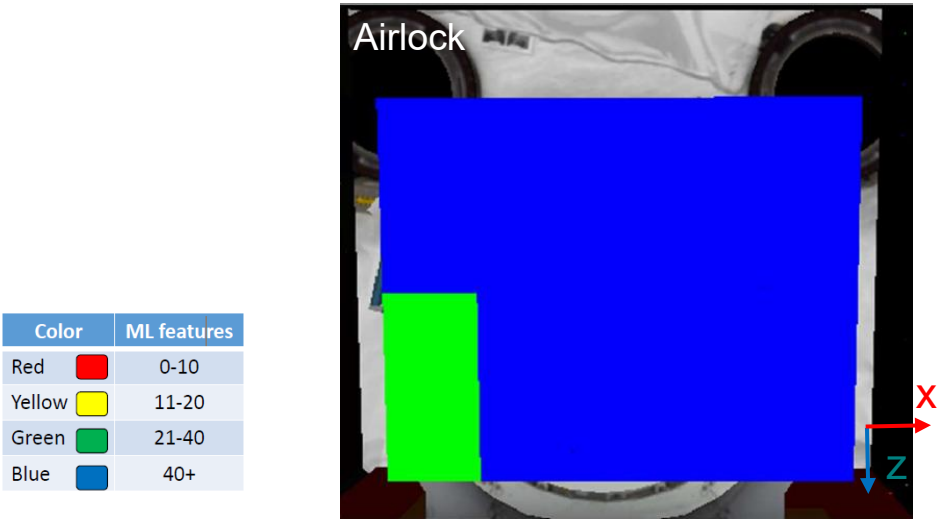


Figure 5-5 ML features on Airlock

5.9. Code review

In the Final Round, before the submitted APK is uplinked to Astrobe on orbit, JAXA / ARC performs a code review for safety confirmation in advance. In the code review, if there is an inappropriate code in the submitted APK, we might delete it or instruct the participants to rewrite it.

To ensure stability, should we confirm that the code may utilize more than three threads simultaneously, we will instruct the participants to rewrite it.

5.10. Setting the application ID

Each Final Round APK must have a unique application ID to avoid conflict when installing on Astrobe in the ISS. The application ID will be specified for the finalists later.

5.11. Questions and information exchange

You can post questions, share programming tips and exchange information with other teams on the bulletin board. Make effective use of it to create your program!

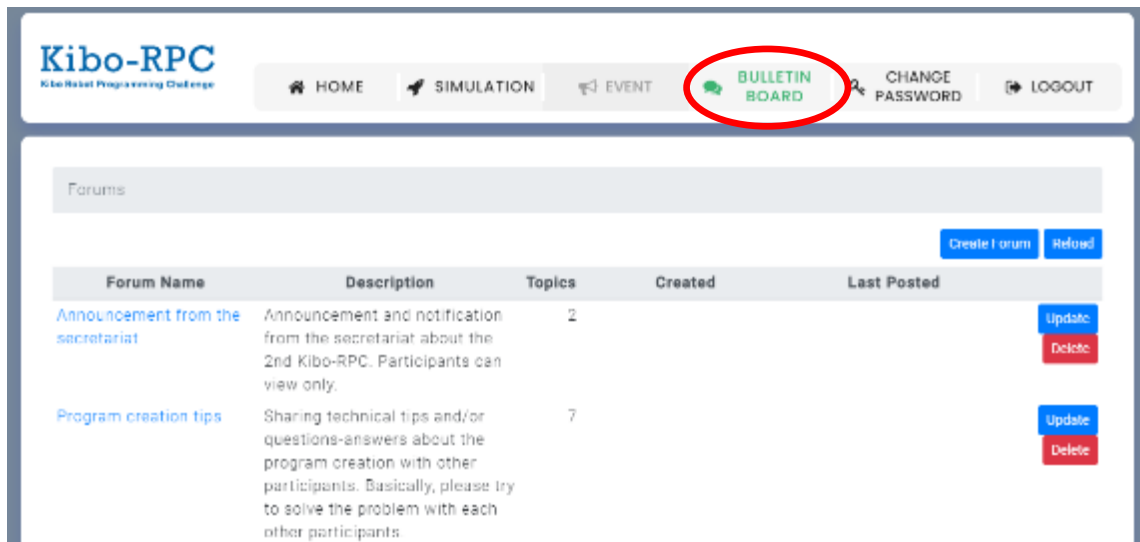


Figure 5-6 BULLETIN BOARD page

5.12. Library for Image Processing

We recommend using OpenCV as the image processing library. The version 'opencv-contrib:4.5.3.0' was used for our verification of normal functioning.

If the contrib module is not required, you can use 'org.opencv:opencv:4.9.0'. When updating the version, please be aware that some functions may have changed.

For detailed information, please refer to the OpenCV documentation.

<https://docs.opencv.org/4.9.0/index.html>

■ Pattern Matching Notes

When utilizing pattern matching with OpenCV, it's important to be mindful of certain considerations.



- The matchTemplate function executes without changing the size of the input template image. You will need to implement image resizing yourself.
- The matchTemplate function operates without changing the angle of the input template image. You will need to implement image rotation yourself.
- Depending on the threshold of similarity, there's a possibility that the same area of the image may be detected multiple times.
- If the target image for pattern matching is large, it can result in longer processing times. Let's consider source codes to reduce processing time.
- The matchTemplate function measures similarity by taking into account the blank areas of the template image. You may want to crop the template image for use.

5.13. Library for Object Detection

We recommend using TensorFlow Lite or OpenCV's DNN module for object detection.

For verification, we have confirmed the functioning of the following libraries:

- 'org.tensorflow:tensorflow-lite-task-vision:0.4.0'
- 'org.tensorflow:tensorflow-lite-support:0.4.3'
 - Key packages:
 - org.tensorflow.lite.Interpreter
 - org.tensorflow.lite.task.vision.detector.ObjectDetector
- 'org.opencv:opencv:4.9.0'
 - Key packages:
 - org.opencv.dnn.Dnn
 - org.opencv.dnn.Net

5.14. Conversion of Object Detection Models Other Than 'tflite' Format

When training with libraries other than TensorFlow Lite, it is necessary to convert the object detection model.

For verification, we used Python for model conversion.

We confirmed the functioning of a model trained in pt format based on YOLOv8n, which was converted to 'tflite' format.

The conversion from pt format to 'tflite' format requires an intermediate conversion to 'onnx' format. (The OpenCV DNN module supports 'onnx'.)

Please refer to the following libraries for the conversion:

- pt → onnx

```
ultralytics
onnx
torch
```

Reference: <https://docs.ultralytics.com/integrations/onnx/#usage>

- onnx → tflite
onnx2tf: <https://pypi.org/project/onnx2tf/>

Model conversion is prone to errors caused by dependencies of each library, so it is recommended to create Docker images for each conversion and separate the execution environments.

5.15. Adding Libraries

If there is a library you wish to use in your program, it is possible to add it to the source code.

To add a library to the source code, please insert the necessary entries in the dependencies section of the <YOUR_APK_PATH>/app/build.gradle file.

```
dependencies {
    implementation project(':guest_science_library')
    implementation project(':kibo_rpc_api')
    implementation fileTree(dir: 'libs', include: ['*.jar'])
    implementation 'com.android.support:appcompat-v7:26.+'
    implementation 'com.android.support.constraint:constraint-layout:1.0.2'
    implementation('org.ros.rosjava_core:rosjava:[0.2,0.3)') {
        exclude group: 'xml-apis'
        exclude group: 'junit'
    }
    implementation('org.ros.android_core:android_10:[0.2,0.3)') {
        exclude group: 'xml-apis'
        exclude group: 'junit'
    }
    implementation 'com.quickbirdstudios:opencv-contrib:4.5.3.0'

    // Additional Library
    implementation 'org.tensorflow:tensorflow-lite-task-vision:0.4.0'
}
```

※The source code provided by the Kibo-RPC secretariat is set to a specific version of the build tool. This version cannot be changed. When adding libraries, please verify in advance whether they are compatible with the currently set version of the build tool. If there is no compatibility, the library may not be usable.

5.16. 【For the Final Round】 About the Noise Simulation Module

For the final round simulator, images taken by NavCam and DockCam may contain two types of noise.

- Gaussian Noise :
 - Whitish, grainy noise may appear in the image.

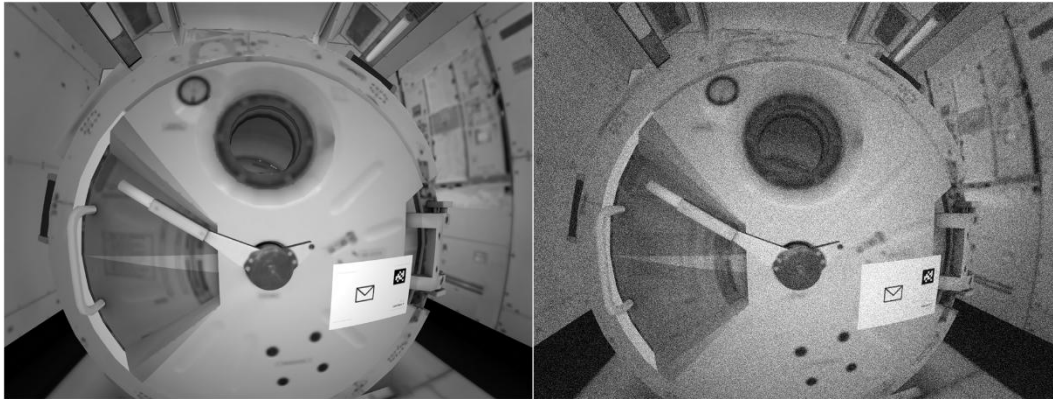


Figure 5-1 Example of Gaussian Noise (Left: Minimum / Right: Maximum)

- Illumination Variation Noise :
 - The entire image or parts of it may become darker or brighter.

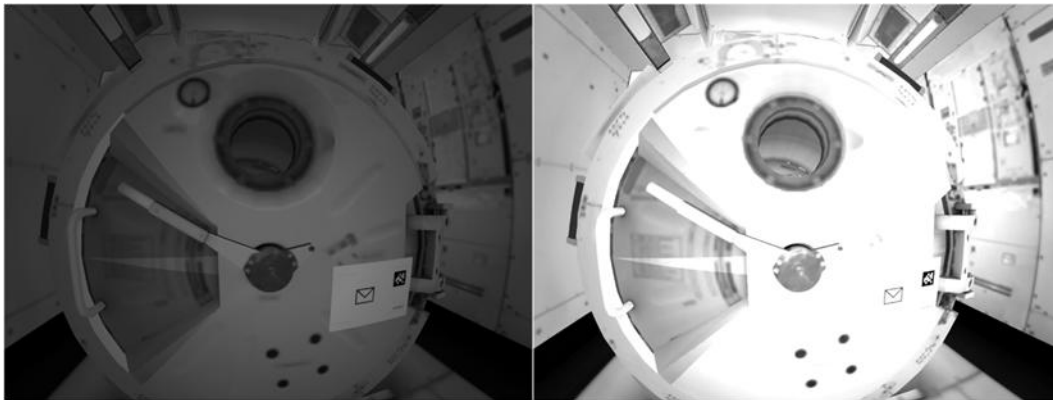


Figure 5-2 Example of Illumination Variation Noise (Left: Darkest / Right: Brightest)

Both types of noise are artificially added to existing NavCam and DockCam images.
The noise parameters are randomly assigned over time.
(Note: The specific parameters are not disclosed.)

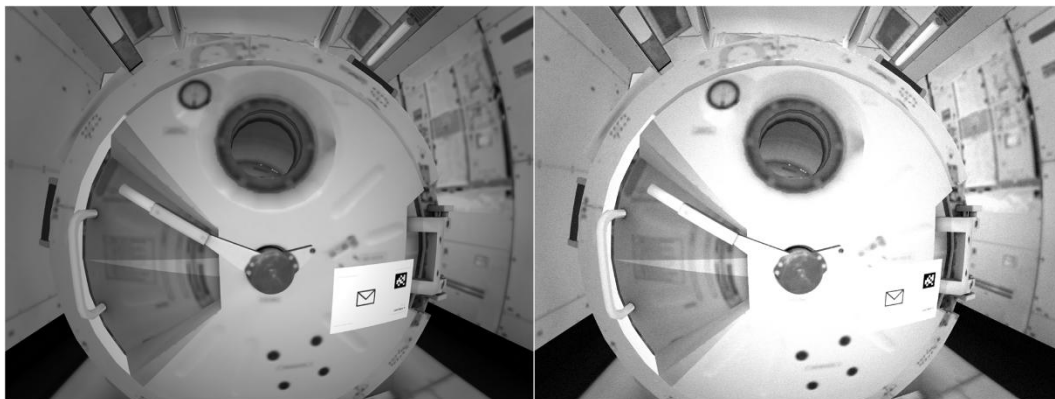


Figure 5-3 Noise Application Example (Left: Original / Right: Randomized)

5.17. 【For the Final Round】 About the Gravity Simulation Module

In the simulator for the final round, microgravity and pulse accelerations simulating Shuttle Docking will occur.

While Astrobees hovering function allows movement to designated positions, please make sure to verify whether image processing and completion reporting are being carried out at the correct locations.

Note that gravity changes occur at a frequency of 1Hz.

reference : [Shuttle Docking](#)

6. Game API details

Details of the Kibo-RPC's game APIs are listed below.

6.1. Method summary

Table 6-1 Method summary

Modifier and Type	Method and Description
gov.nasa.arc.astrobee.Result	flashlightControlBack(float brightness) Set the brightness of the back flash light.
gov.nasa.arc.astrobee.Result	flashlightControlFront(float brightness) Set the brightness of the front flash light.
Bitmap	getBitmapDockCam() Gets Bitmap image of DockCam.
Bitmap	getBitmapNavCam() Get Bitmap image of NavCam.
double[][]	getDockCamIntrinsics() Get camera matrix and distortion coefficients of DockCam.
org.opencv.core.Mat	getMatDockCam() Gets Mat image of DockCam.
org.opencv.core.Mat	getMatNavCam() Gets Mat image of NavCam.
double[][]	getNavCamIntrinsics() Get camera matrix and distortion coefficients of NavCam.
gov.nasa.arc.astrobee.Kinematics	getRobotKinematics() Gets current data related to positioning and orientation for Astrobee.
gov.nasa.arc.astrobee.Result	moveTo(gov.nasa.arc.astrobee.types.Point goalPoint, gov.nasa.arc.astrobee.types.Quaternion orientation, boolean printRobotPosition) Moves Astrobee to the given point and rotates it to the given orientation.

Modifier and Type	Method and Description
void	notifyRecognitionItem() Notify that Astrobees has recognized the TargetItem held by the astronaut.
gov.nasa.arc.astrobees.Result	relativeMoveTo(gov.nasa.arc.astrobees.types.Point goal Point, gov.nasa.arc.astrobees.types.Quaternion orientation, boolean printRobotPosition) Moves Astrobees to the given point using a relative reference and rotates it to the given orientation.
boolean	reportRoundingCompletion() Report rounding completion and blink the lights.
void	saveBitmapImage(Bitmap image, java.lang.String imageName) Save a bitmap image for debug.
void	saveMatImage(org.opencv.core.Mat image, java.lang.String imageName) Save a mat image for debug.
void	setAreaInfo(int areald, java.lang.String itemName) Set matching of an area and an item.
void	setAreaInfo(int areald, java.lang.String itemName, int number) Set matching of an area and an item, and the number of an item.
void	shutdownFactory() This method shutdown the robot factory in order to allow java to close correctly.
boolean	startMission() Undock Astrobees from docking station.
void	takeTargetItemSnapshot() Take a snapshot of TargetItem. When execute this method, the mission is completed.

6.2. Method details

- **getRobotKinematics**

```
public Kinematics getRobotKinematics()
```

Gets current data related to positioning and orientation for Astrobe. Note that the data cannot be trusted when the confidence is POOR or LOST.

Returns:

Current Kinematics.

- **getBitmapNavCam**

public Bitmap getBitmapNavCam()

Gets Bitmap image of NavCam.

Returns:

Bitmap image of NavCam(1280 px x 960 px) or null if an internal error occurs.
Format:Bitmap.Config.ARGB_8888

- **getBitmapDockCam**

public Bitmap getBitmapDockCam()

Gets Bitmap image of DockCam.

Returns:

Bitmap image of DockCam(1280 px x 960 px) or null if an internal error occurs.
Format:Bitmap.Config.ARGB_8888

- **getMatNavCam**

public Mat getMatNavCam()

Gets Mat image of NavCam.

Returns:

Mat image of NavCam(1280 px x 960 px) or null if an internal error occurs. Format:CV8UC1

- **getMatDockCam**

public Mat getMatDockCam()

Gets Mat image of DockCam.

Returns:

Mat image of DockCam(1280 px x 960 px) or null if an internal error occurs. Format:CV8UC1

- **flashlightControlFront**

public Result flashlightControlFront(float brightness)

Set the brightness of the front flash light.
In the final round, brightness greater than 0.01f will be reset to 0.01f.

Parameters:

brightness - Brightness percentage between 0 - 1.

Returns:

A Result instance carrying data related to the execution. Returns null if the command is NOT executed because of an error



- **flashlightControlBack**

```
public Result flashlightControlBack(float brightness)
```

Set the brightness of the back flash light.

In the final round, brightness greater than 0.01f will be reset to 0.01f.

Parameters:

brightness - Brightness percentage between 0 - 1.

Returns:

A Result instance carrying data related to the execution. Returns null if the command is NOT executed because of an error.

- **moveTo**

```
public Result moveTo(Point goalPoint,
                    Quaternion orientation,
                    boolean printRobotPosition)
```

Moves Astrobees to the given point and rotates it to the given orientation.

Parameters:

goalPoint - Absolute cardinal point (xyz)

orientation - An instance of the Quaternion class. You may want to use CENTER_US_LAB or CENTER_JEM as an example depending on your initial position.

printRobotPosition - Flag whether to print robot positions in log or not.

Returns:

A Result instance carrying data related to the execution. Returns null if the command is NOT executed because of an error.

- **relativeMoveTo**

```
public Result relativeMoveTo(Point goalPoint,
                             Quaternion orientation,
                             boolean printRobotPosition)
```

Moves Astrobees to the given point using a relative reference and rotates it to the given orientation. This api can take up to 30 seconds when localization is not accurate enough.

Parameters:

goalPoint - The relative end point (relative to Astrobees)

orientation - The absolute orientation

printRobotPosition - Flag whether to print robot positions in log or not.

Returns:

A Result instance carrying data related to the execution. Returns null if the command is NOT executed because of an error.

- **startMission**

```
public boolean startMission()
```

Undock Astrobees from docking station. Astrobees then starts counting the mission time.

Returns:



Returns True if the execution is successful. Returns false if the command is NOT executed because of an error.

- **notifyRecognitionItem**

```
public void notifyRecognitionItem()
```

Notify that Astrobees has recognized the TargetItem held by the astronaut.

- **reportRoundingCompletion**

```
public boolean reportRoundingCompletion()
```

Report rounding completion and blink the lights.

Returns:

Returns True if the execution is successful. Returns false if the command is NOT executed because of an error.

- **takeTargetItemSnapshot**

```
public void takeTargetItemSnapshot()
```

Take a snapshot of TargetItem.

When execute this method, the mission is completed.

- **getNavCamIntrinsics**

```
public double[][] getNavCamIntrinsics()
```

Get camera matrix and distortion coefficients of NavCam. Different values are returned on orbit and in the simulator. *The parameter values are different between the simulator and real Astrobees in the ISS as shown below. Therefore this API returns different value depending on whether the APK is running on simulator or on real Astrobees.

Returns:

Array of camera parameters [camera matrix, distortion coefficients] for NavCam. The array of the camera matrix and distortion coefficients is as follows.

- Simulator

Camera matrix: [523.105750, 0.000000, 635.434258, 0.000000, 534.765913, 500.335102, 0.000000, 0.000000, 1.000000]

Distortion coefficients: [-0.164787, 0.020375, -0.001572, -0.000369, 0.000000]

- Real Astrobees

Camera matrix: [608.8073, 0.0, 632.53684, 0.0, 607.61439, 549.08386, 0.0, 0.0, 1.0]

Distortion coefficients: [-0.212191, 0.073843, -0.000918, 0.001890, 0.0]

- **getDockCamIntrinsics**

```
public double[][] getDockCamIntrinsics()
```

Get camera matrix and distortion coefficients of DockCam. Different values are returned on orbit and in the simulator. *The parameter values are different between the simulator and real Astrobe in the ISS as shown below. Therefore this API returns different value depending on whether the APK is running on simulator or on real Astrobe.

Returns:

Array of camera parameters [camera matrix, distortion coefficients] for DockCam. The array of the camera matrix and distortion coefficients is as follows.

- Simulator

Camera matrix: [661.783002, 0.000000, 595.212041, 0.000000, 671.508662, 489.094196, 0.000000, 0.000000, 1.000000]

Distortion coefficients: [-0.215168, 0.044354, 0.003615, 0.005093, 0.000000]

- Real Astrobe

Camera matrix: [753.51021, 0.0, 631.11512, 0.0, 751.3611, 508.69621, 0.0, 0.0, 1.0]

Distortion coefficients: [-0.411405, 0.177240, -0.017145, 0.006421, 0.000000]

- **saveBitmapImage**

```
public void saveBitmapImage(Bitmap image,
                             java.lang.String imageName)
```

Save a bitmap image for debug. The maximum pixel size of an image is 1228800 (height x width) and up to 50 images can be saved per simulation. The image is saved in Android Emulator (/sdcard/data/) as a png file and can be download on the dashboard display.

Parameters:

image - Bitmap Images to save.

imageName - string Image name to save.

- **saveMatImage**

```
public void saveMatImage(Mat image,
                          java.lang.String imageName)
```

Save a mat image for debug. The maximum pixel size of an image is 1228800 (height x width) and up to 50 images can be saved per simulation. The image is saved in Android Emulator (/sdcard/data/) as a png file and can be download on the dashboard display.

Parameters:

image - Mat Images to save.

imageName - string Image name to save.

- **setAreaInfo**

```
public void setAreaInfo(int areald,
    java.lang.String itemName)
```

Set matching of an area and an item.

Parameters:

areald - area ID

itemName - item name

- **setAreaInfo**

```
public void setAreaInfo(int areald,
    java.lang.String itemName,
    int number)
```

Set matching of an area, an item, and the number of an item.

Parameters:

areald - area ID

itemName - item name

number - item number

6.2.1. Type information

Please refer the following links for information about Types implemented in astrobee_android.

Table 6-2 Type information

Type	URL
gov.nasa.arc.astrobee.Kinematics	https://github.com/nasa/astrobee_android/blob/a8560ab0270ac281d8eadeb48645f4224582985e/astrobee_api/api/src/main/java/gov/nasa/arc/astrobee/Kinematics.java
gov.nasa.arc.astrobee.Result	https://github.com/nasa/astrobee_android/blob/a8560ab0270ac281d8eadeb48645f4224582985e/astrobee_api/api/src/main/java/gov/nasa/arc/astrobee/Result.java
gov.nasa.arc.astrobee.types.Vec3d	https://github.com/nasa/astrobee_android/blob/a8560ab0270ac281d8eadeb48645f4224582985e/astrobee_api/api/src/main/java/gov/nasa/arc/astrobee/types/Vec3d.java
gov.nasa.arc.astrobee.types.Quaternion	https://github.com/nasa/astrobee_android/blob/a8560ab0270ac281d8eadeb48645f4224582985e/astrobee_api/api/src/main/java/gov/nasa/arc/astrobee/types/Quaternion.java
gov.nasa.arc.astrobee.types.Point	https://github.com/nasa/astrobee_android/blob/a8560ab0270ac281d8eadeb48645f4224582985e/astrobee_api/api/src/main/java/gov/nasa/arc/astrobee/types/Point.java

7. Tips for Astrobees Characteristics

The for	Name	Explanation	tips
	HazCam	A monochrome camera for detecting obstacles within 30 cm	
	NavCam	A monochrome camera for image data processing and taking a photo after sending finish command	
	SciCam	A color camera for taking a video (The participants cannot utilize this.)	
	DockCam	A monochrome camera for docking to the docking station	
	PerchCam	A monochrome camera for grabbing a handrail	
	Flashlight	Use this when reading QR code	
	Speaker	Playing audio files. In the 4 th Kibo-RPC, the speaker will not be used.	

Astrobees characteristics will be provided in this section to be considered for successful runs.

7.1. Rendering of Astrobees

Figure 8-1 indicates Astrobees is equipped with some external hardware components. The hardware surrounded with one-foot cube (about 32 cm wide) are used in the simulator and ISS Final Round.

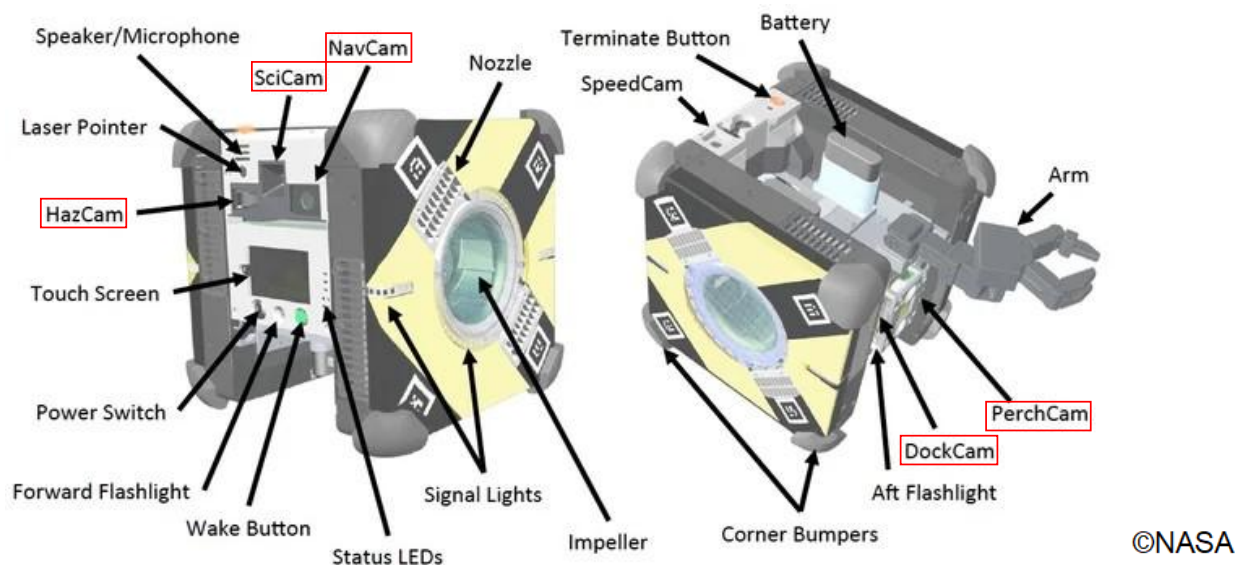


Figure 7-1 External hardware components

Table 7-1 External hardware components

Figures and tables below show the positions of the hardware components.

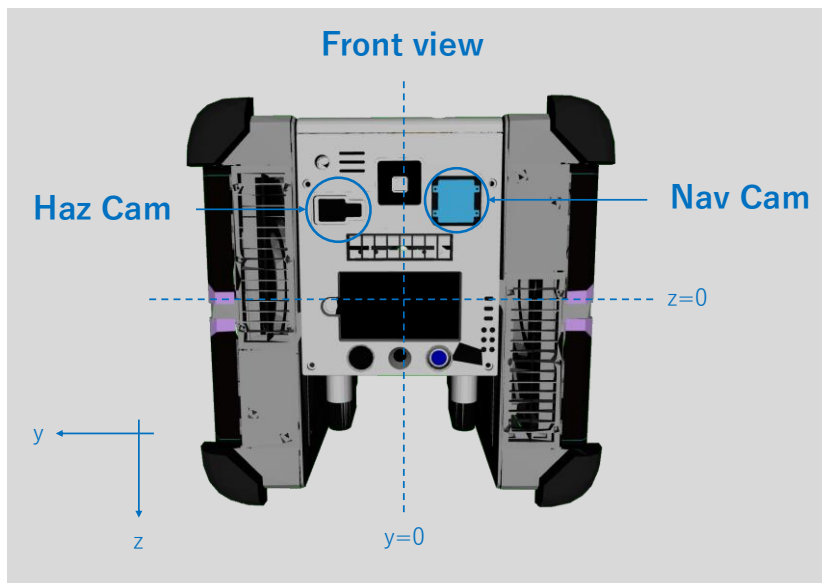


Figure 7-2 Astrobe Front View

Table 7-2 Distances from center point

	x[m]	y[m]	z[m]
Nav Cam	0.1177	-0.0422	-0.0826
Haz Cam	0.1328	0.0362	-0.0826

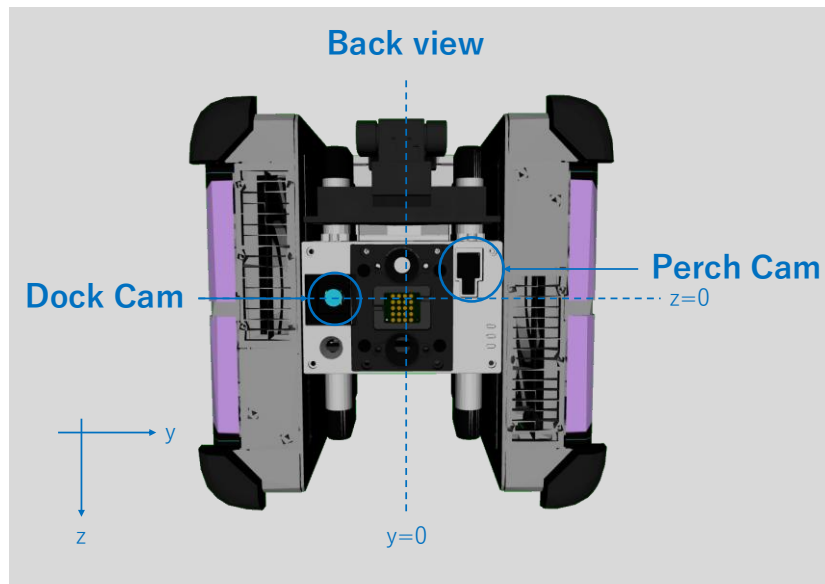


Figure 7-3 Astrobee Back View

Table 7-3 Distances from center point

	x[m]	y[m]	z[m]
Dock Cam	-0.1061	-0.054	-0.0064
Perch Cam	-0.1331	0.0509	-0.0166

7.2. Specification of Astrobee

- Mass: 10kg (Installed only two of the four batteries)
- Maximum Velocity: 0.5 m/s
- Maximum Thrust (X axis): 0.6 N
- Minimum moving distance: 0.05 m.
- Minimum rotating angle: 7.5 degrees.
- If the Astrobee detects the actual obstacles in front, it will automatically stop and then maintain its position and orientation. The moving path it is on will also be discarded.

7.3. References of Astrobee

- GitHub-1 (<https://github.com/nasa/astrobee>)
- GitHub-2 (https://github.com/nasa/astrobee_android)
- Website of Astrobee (<https://www.nasa.gov/astrobee>)